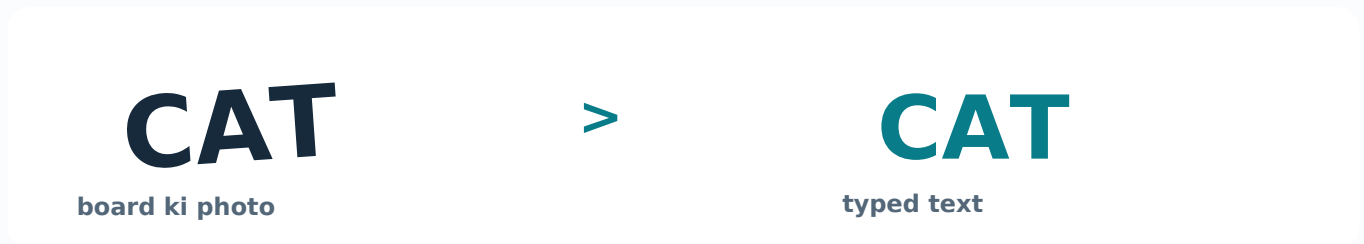


Ek hi CAT photo, start se end tak

Pichhle step se: Hum bilkul shuru par hain: board ki photo mein CAT likha hai.

Humein banana kya hai?

Photo mein word dikhna aur computer ke paas typed word hona alag hai. Goal: photo do aur aisa **CAT** text mile jise copy, search aur edit kar sakein. Photo se text padhne ke poore kaam ko **OCR** kehte hain.



PARSeq ka simple role

PARSeq ek text-reading model ka naam hai. Model ka matlab learned calculations wala system. Usko text wala image area do, woh characters predict karega. Abhi encoder, attention ya equations yaad nahi karne.

Kitab ki continuity

Har page par wahi CAT image aage chalegi. Kabhi chhote made-up numbers se calculation samjhayenge; unhe actual measured model values mat samajhna. Har page ka end batayega: ab kya bana aur next step kyun chahiye.

Ab hamare paas: Photo abhi photo hi hai; typed CAT abhi nahi nikla.

Agla page: Full photo se CAT wali jagah nikaalenge.

Pehle CAT wali jagah nikaalo

Pichhle step se: Hamare paas board ke saath background wali full photo hai.

Kahan likha hai, phir kya likha hai

Text ki jagah dhoondhna **detection** hai. Us jagah ka text padhna **recognition** hai. PARSeq recognition wala part hai. Full OCR system mein detection ke liye alag part ho sakta hai.

Full photo

>

Text ki jagah

>

CAT ka crop

CAT ka crop

Selected rectangular image area nikaalna **crop** karna hai. Ab board se CAT wala rectangle model ko denge. Agar T ka aadha hissa crop ke bahar reh gaya, toh reading model ko incomplete information milegi.

Is decision ka effect

Bahut background ya cut letters recognition ko difficult bana sakte hain. Tedhi image ko seedha rotate karna **deskew** hai; shape ko readable rectangle mein adjust karna **rectification**. Har crop par inki need same nahi.

Ab hamare paas: Ek chhoti image jismein complete C, A, T dikh rahe hain.

Agla page: Ab dekhenge ki computer ko is crop se kya numbers milte hain.

Khud check: Crop ne T ka half kaat diya. Pehla check model weights honge?

[Jawab >](#)

CAT ki image ab colour numbers hai

Pichhle step se: Complete CAT crop mil gaya; computer ko ise calculate karne layak form mein lena hai.

Pixel ka matlab

Photo ka ek chhota colour point **pixel** hai. Toy black-and-white drawing mein 0 dark aur 1 light maan lo. Real colour image mein red, green, blue ke values hote hain; in teen colour layers ko **RGB channels** kehte hain.



Dark squares milkar C jaisi shape banate hain. Real model ko RGB numbers milte hain.

C ki shape kaise stored hai

Dark pixels ki arrangement C jaisi dikhti hai. Computer ko starting par numbers milte hain, typed C nahi. Dusre font, angle ya lighting mein same C ke pixel numbers alag ho sakte hain.

Size ko fixed rakhna

Common PARSeq setup crop ko height 32, width 128 pixels par resize karta hai.

Resize matlab pixel dimensions change karna. Is stage par hum image ko model ki expected size mein taiyaar kar rahe hain.

Ab hamare paas: CAT ka resized 32 x 128 RGB crop, ab numerical colour values ke form mein.

Agla page: In numbers ko organised dabbe mein rakhenge.

Tensor: numbers ka organised dabba

Pichhle step se: CAT crop ke red, green, blue values aur 32 x 128 pixel positions available hain.

Dabba kitna bada hai?

Numbers ko dimensions ke saath arrange karo; ise **tensor** kehte hain. Dimensions ki ginti likhna uski **shape** hai. Ek image ke liye [1,3,32,128]: 1 crop, 3 colour channels, height 32, width 128.

CAT input: [1, 3, 32, 128]

Total values = $1 \times 3 \times 32 \times 128 = 12,288$. Yeh letters ki count nahi.

Batch aur layout

Batch = ek saath process hone wali images ka group; uski ginti B hai. Input [B,3,32,128]. **Layout** numbers ki directions ka order hai. NCHW mein N=batch, C=channels, H=height, W=width. NHWC mein channel direction last hai.

Colour scale bhi match karo

Reference pixel values ko pehle 0 se 1 par laata hai, phir $(x-0.5)/0.5$ karta hai. Is scale adjustment ko **normalization** kehte hain: 0 -> -1, 0.5 -> 0, 1 -> 1. Phone aur computer par same rule chahiye.

Ab hamare paas: Ek correctly sized, correctly ordered, normalized CAT input tensor.

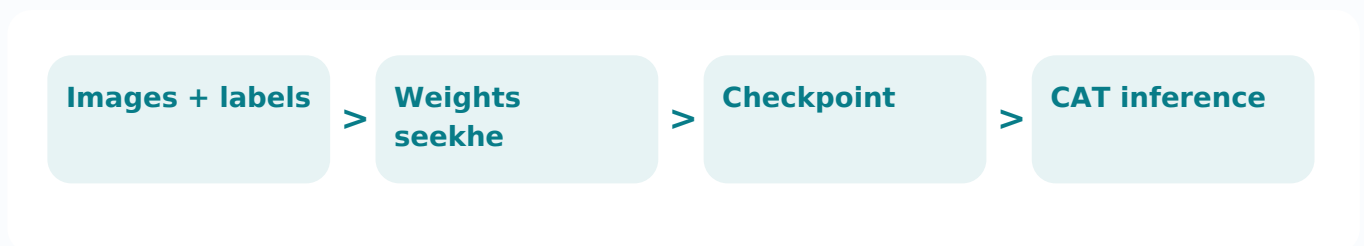
Agla page: Ab samjhenge ki model ke rules seekhe hue kahan se aaye.

Calculations ko C pehchanna kisne sikhaya?

Pichhle step se: CAT ka input tensor ready hai; lekin colours ko letters se jodne ki ability chahiye.

Examples aur correct answers

Model ko training mein CAT ki alag photos aur correct text CAT diya jaata hai. Image ke correct text ko **label** kehte hain. Examples se internal adjustable numbers seekhna **training** hai.



Weights vs current image ka work

Saved adjustable numbers **weights / parameters** hain. Current CAT image par beech mein nikle temporary numbers **activations** hain. Weights model ki learned ability carry karte hain; activations abhi wali image ki calculations hain.

Ab hum read kar rahe hain

Trained model ki saved file **checkpoint** hai. Nayi CAT photo ko fixed learned weights se padhna **inference** hai. Abhi hum trained checkpoint use kar rahe hain. Weights update karne ki training story baad mein return karegi.

Ab hamare paas: CAT tensor ke saath trained weights hain; ab meaningful image processing ho sakti hai.

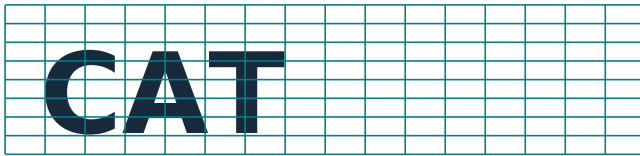
Agla page: Image ko small pieces mein organise karenge.

CAT ko patches mein baanto

Pichhle step se: Input 32 x 128 RGB tensor aur trained model available hain.

Patch kya hai?

Image ka ek small rectangle **patch** hai. Common setup mein patch height 4, width 8 pixels. Height mein $32/4 = 8$ rows; width mein $128/8 = 16$ columns. Total $8 \times 16 =$ **128 patches**.



CAT par 8 rows aur 16 columns.
128 image patches

Ek patch ke numbers

Har patch mein $4 \times 8 \times 3 = 96$ colour values hain. Inhe ek list mein likhna **flatten** karna hai. Numbers ki ek list **vector** kehlaati hai. Ab har patch ke liye 96-number vector samajh sakte ho.

Patch ka matlab character nahi

C kai patches mein fail sakta hai. Ek patch mein background aur A ka ek stroke, yani line ka piece, dono ho sakte hain. Isliye 128 patches ka matlab 128 letters nahi. Letters abhi decide nahi hue.

Ab hamare paas: 128 patch vectors; har vector mein raw 96 colour values.

Agle page: In raw vectors ko learned visual representation mein badlenge.

Khud check: 128 patches ban gaye. Kya 128 characters mil gaye?

[Jawab >](#)

Patch ke numbers se naya vector banao

Pichhle step se: Har CAT patch ki 96 colour values wali list hai. Yeh abhi raw colour information hai.

Badalne ka matlab concrete example se

Asli 96 values ki jagah toy list $x=[1,0,1]$ lo. Maan lo learned rule hai:
 $output1=2x1+x2$; $output2=x2-x3$. $x1$ first value, $x2$ second, $x3$ third. Result **[2,-1]**.
Yeh letter C predict nahi hua; bas naya numerical representation bana.

Toy: [1,0,1] -> learned rule -> [2,-1]

Actual Small: 96 values -> 384 values, har patch ke liye.

Actual PARSeq mein kya hai

Learned transformation ko **projection** kehte hain. Common Small model har 96-number patch ko **384-number vector** mein map karta hai. Length ko D likhenge; Small mein $D=384$. Is representation ko patch **embedding** ya visual **token** bolenge.

Yeh step kyun useful hai?

Weights training se aise seekhte hain ki naye vectors reading ke kaam ki information carry karein. Ek dimension ka fixed meaning sirf curve ho, yeh zaroori nahi. Har patch par same learned rule lagta hai; content badalne se output vectors badalte hain.

Ab hamare paas: 128 visual-token vectors, har ek mein 384 values. Ab bhi final CAT text nahi.

Agla page: Har vector ke saath uski image position ki pehchaan jodenge.

Vector ko uski jagah bhi batao

Pichhle step se: 128 patch embeddings hain; har patch ki appearance represented hai.

Same pattern alag jagah ho sakta hai

CAT mein left-side curve aur right-side line ki locations important hain.

Appearance-only vectors se model ko original grid position directly label nahi hoti. Har patch ke vector mein us location ke learned numbers add karte hain.

$$[2, -1] + [0.3, 0.5] = [2.3, -0.5]$$

Toy 2-number example; actual Small mein 384-number vectors add hote hain.

Ek addition example

Toy patch vector $[2, -1]$ aur us location ka learned vector $[0.3, 0.5]$ lo. Add karne par $[2.3, -0.5]$. Location vector ko **position embedding** kehte hain. Actual model mein dono vectors length 384 ke hain.

Exactly kya add hua?

Vector ab appearance aur location dono ki information carry kar sakta hai. Location vector mein yeh nahi likha ki yahan C hai. Yeh image patch ka address-signal hai. Baad mein output letter positions ka alag address-signal aayega.

Ab hamare paas: 128 vectors jo patch content ke saath patch location bhi represent karte hain.

Agla page: Ab ek patch ko doosre patches ki useful information lene denge.

Attention: ek patch ko extra clue do

Pichhle step se: Har patch ka location-aware vector hai; abhi har vector mainly apne area se shuru hua hai.

CAT ke ek area ki limitation

A ka ek chhota patch sirf slant line dikhaye toh full character clear nahi. Doosra patch A ki crossbar, yani beech wali line, ka visual information de sakta hai. Model ko related areas ke clues combine karne ka way chahiye.

Source u: [2.3, -0.5]

75% contribution

Mix: [1.725, 0.625] -> richer vector

Source v: [0, 4]

25% contribution

Mix ka actual calculation

Pichhle page wala vector $u=[2.3, -0.5]$ lo. Doosre patch ka toy vector $v=[0, 4]$ hai. Target ke liye weights 0.75 aur 0.25 hue. $Mix = 0.75u + 0.25v = [1.725, 0.625]$. Information weighted combine hui; sirf ek source ki copy nahi bani.

Naam ab samjho

Relevant sources ki numerical information ko weighted mix karne ka operation **attention** hai. Yahan weight ka meaning contribution hai; answer correct hone ka 75% chance nahi. Actual CAT image ke vectors aur weights learned calculations se aate hain; yeh toy numbers sirf process dikhate hain.

Ab hamare paas: Target patch ko doosre patches ki information se richer vector mil sakta hai.

Agla page: Ab dekhenge model 0.75 aur 0.25 jaise weights banata kaise hai.

Khud check: Attention ka 0.75 weight kya C correct hone ka 75% chance hai?

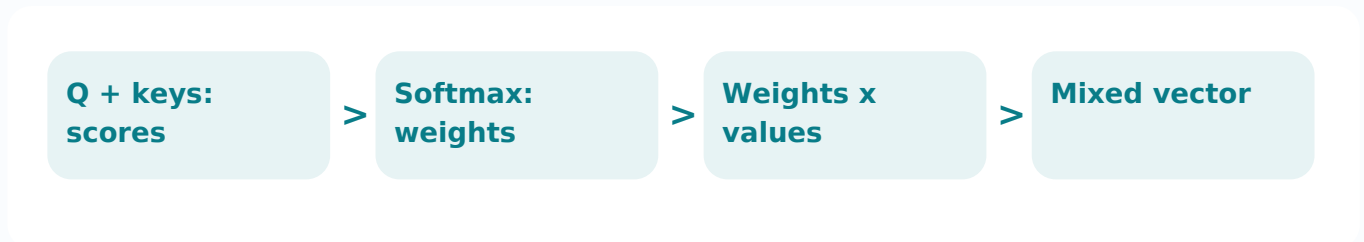
[Jawab >](#)

Q, K, V: weights kaise decide hue?

Pichhle step se: Attention information mix karta hai; mix se pehle har source ki relevance score karni hai.

Teen numerical roles

Target ke vector se **query Q** banti hai: kis kind ki information request hai. Sources se **keys K** banti hain: matching signals. Sources se **values V** banti hain: actual mix hone wali information. Q,K,V learned projections ke output vectors hain.



Matching se contribution tak

Q aur har K ki numerical matching se raw **scores** bante hain. Scores ko positive weights mein convert karne ke rule ko **softmax** kehte hain; weights sum 1 hote hain. Toy scores $\ln(3), 0$ se weights $0.75, 0.25$ milte hain. In natural logarithm function hai; formula yaad karna abhi zaroori nahi.

Previous page complete karo

Pichhle page ke example mein V ko source vectors $[2.3, -0.5]$ aur $[0, 4]$ ke barabar maan kar weights se mix kiya: $[1.725, 0.625]$. Actual model mein V learned projection se banta hai. Q/K decide karte hain contribution; V contribute karta hai information. Actual attention mein score scaling bhi hoti hai, taaki values suitable numerical range mein rahein.

Ab hamare paas: CAT patch features ke beech relevant information sharing ka complete operation.

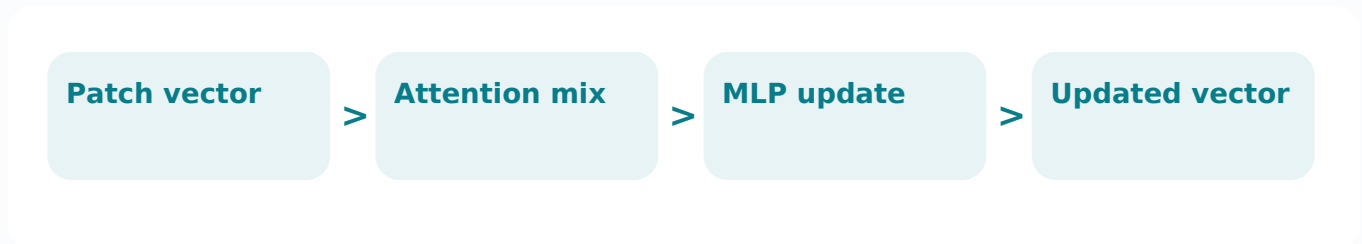
Agla page: Is sharing ke saath token-wise processing jodkar encoder layers banengi.

Ek processing block ka poora kaam

Pichhle step se: Patch vectors attention se doosre patches ki information le sakte hain.

Self-attention aur multiple views

Same patch group ke tokens aapas mein information lein toh **self-attention**. Attention ka ek learned view **head** hai. Multiple heads, yani **multi-head attention**, different learned views parallel calculate karte hain.



Mix ke baad vector ko process karo

Har vector par chhota learned calculation network lagta hai: **MLP**. Input ko update ke saath add karne ka shortcut **residual connection** hai. Values ka scale adjust karne ka token-wise operation **LayerNorm** hai; before branch lage toh **Pre-LN**.

Block ko repeat kyun karte hain

Attention, MLP aur unke helper operations ka group **Transformer block / layer** hai. CAT ki representation successive blocks se update hoti hai. **ViT**, Vision Transformer, image tokens par aise blocks use karta hai. Common Small encoder mein 12 layers aur 6 heads per layer hain.

Ab hamare paas: Patches repeated visual processing ke baad more useful vectors hain.

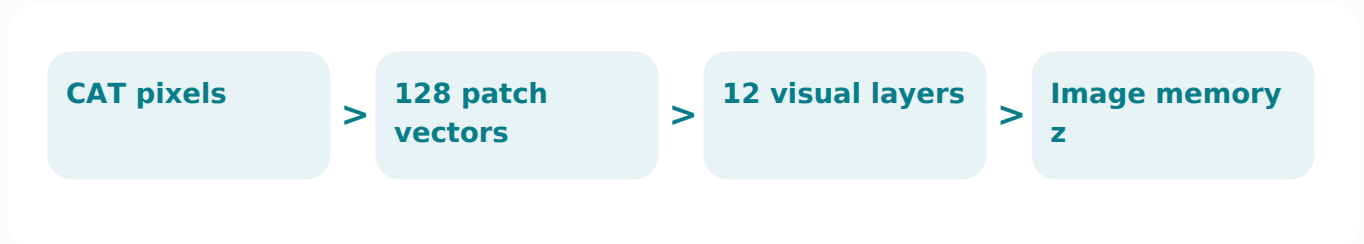
Agla page: Ab in processed vectors ko encoder output ki ek exact shape mein dekhenge.

Encoder output: exactly kya mila?

Pichhle step se: CAT ke 128 location-aware patch vectors, 12 processing layers se update ho chuke hain.

Encoder naam ka scope

Pixels -> patch embeddings -> positions -> visual processing: is part ko **image encoder** kehte hain. Encoder ka output ab bhi vectors hai. Final typed C,A,T abhi decide nahi hue.



Exact output shape

Small setup mein output **[1,128,384]**: 1 crop, 128 visual tokens, har token 384 feature values. **Feature** useful learned numerical information hai. Is output ko **image memory z** bhi bolte hain; z sirf variable ka naam hai.

What changed from raw input?

Input [1,3,32,128] pixel colours tha. Output [1,128,384] processed visual representation hai. Number count same rehna required nahi. Memory yahan encoder output hai; phone ki RAM ka general term nahi. Ab next part isi z se letters nikaalega.

Ab hamare paas: CAT ki image memory $z = [1,128,384]$, decoder ko dene ke liye ready.

Agla page: Text output ki positions banayenge: first, second, third aur stop.

Khud check: [1,128,384] mein 128 ka matlab 128 predicted letters?

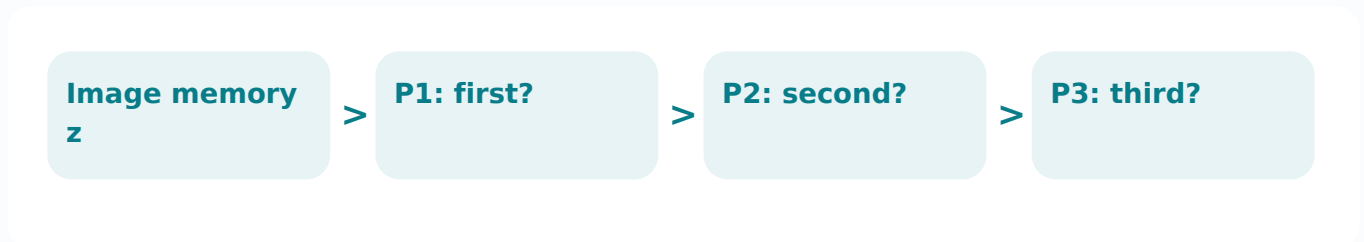
[Jawab >](#)

Output slots: text kahan likha jaayega?

Pichhle step se: Encoder ne CAT image memory z banayi. Ab output text ke positions decide karne hain.

CAT ke liye positions socho

Slot 1 ko first character, slot 2 ko second, slot 3 ko third chahiye. **Slot** bas output ki jagah hai. Us jagah ki learned vector pehchaan **position query** hai: P1, P2, P3 kahenge.



P2 mein A already nahi hai

P2 ka meaning hai second output position ka answer maango. Ismein correct A pehle se stored nahi. Image memory aur available text information se A ki prediction banegi. P2 image ke second patch ya second pixel column ka fixed address nahi.

Maximum positions

Common maximum text length 25 characters hai. Stop signal ke liye extra slot, total up to 26. CAT short hai, lekin same setup longer words ko bhi handle karta hai. Final text banate waqt stop signal determine karega kitne slots useful hain.

Ab hamare paas: Visual memory ke saath output-position queries available hain.

Agla page: Letters aur start/stop symbols ko numeric IDs denge.

C, A, T aur start/stop ke IDs

Pichhle step se: Output positions hain; model ko possible answers ki list aur numeric representation chahiye.

Token aur ID

Text ki ek unit **token** hai; PARSeq aam taur par character units use karta hai. Har token ka numeric identifier **ID** hai. Supported ordinary characters ki list **charset** hai. Token-ID order checkpoint ke expected order se match karna chahiye.

BOS: start

>

C

>

A

>

T + EOS

Teen special signals

BOS = Beginning Of Sequence, start/null context. **EOS** = End Of Sequence, word khatam. **PAD** = padding token, unused group positions bharnе ke liye. BOS input context hai; EOS predicted output ho sakta hai; PAD actual text nahi.

94, 95, 97 ka reason

94 ordinary characters setup mein output classifier 95 classes score karta hai: 94 + EOS. Input embeddings BOS aur PAD bhi handle karti hain, total 97 IDs. **Class** possible answer category hai. CAT ka desired output C,A,T,EOS; desired starting context BOS hai.

Ab hamare paas: Token IDs aur special signals ka meaning clear; decoder ab symbols use kar sakta hai.

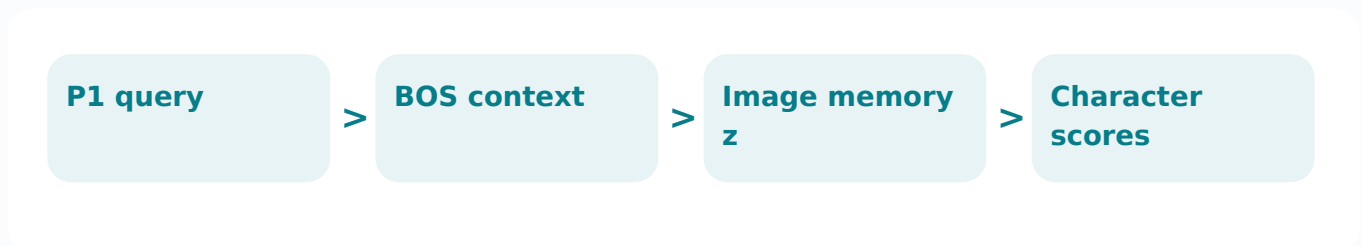
Agla page: Decoder image aur available text ko kaise combine karta hai, woh dekhenge.

Decoder ko do sources milte hain

Pichhle step se: Image memory z , output queries P1/P2/P3 aur text token IDs available hain.

Text source kya hai?

Context woh information hai jo prediction ke waqt available ho. First letter ke liye BOS. Agar C predict ho gaya, second letter ko BOS,C context mil sakta hai. C ki ID ko learned vector mein map karna character embedding hai; text position bhi represent hoti hai.



Decoder ke andar sequence

Decoder character-output banane wala part hai. Position query pehle allowed text context se attention leti hai. Phir image memory z se attention leti hai. Do groups ke beech attention ko **cross-attention** kehte hain. MLP updated vector process karta hai.

P1 ka concrete case

P1 text side par sirf BOS read karta hai. Image side par CAT ke processed visual vectors se evidence leta hai. Final vector se possible symbols ke scores banenge. BOS minimal context hai, image still available hai. Isliye first letter bhi predict ho sakta hai.

Ab hamare paas: First output slot ke liye processed decoder vector; ab ise C/A/T/EOS scores mein badlenge.

Agla page: Raw scores ko choices mein convert karenge.

Scores se first character C choose karo

Pichhle step se: Decoder ka P1 output vector hai; possible character list already defined hai.

Classifier aur logits

Vector se har possible answer ka score banane wala learned part **classifier / output head** hai. Softmax se pehle raw numbers **logits** hain. Character softmax unhe classes ke probability-like scores mein badalta hai, sum 1.

P1 toy scores: C .80 | A .10 | T .05 | EOS .05

Highest class C -> choose C ID -> first output letter C.

Mini choice example

Sirf samjhane ke liye four choices lo: C=0.80, A=0.10, T=0.05, EOS=0.05. Actual 94-character setup mein 95 scores hain. Toy mein highest C hai, isliye C ki ID choose karenge. Highest-score index nikaalne ka rule **argmax** hai.

Attention weights se farq

Attention ka 0.75 source contribution tha. Yahan 0.80 output class ka model-assigned score hai. Dono alag operations ke softmax ho sakte hain. High character score bhi real-world answer correctness ki calibrated guarantee nahi; quality later measure karenge.

Ab hamare paas: First predicted character C. Image memory z reuse ke liye available hai.

Agla page: Ab isi run mein A, T aur EOS nikaalenge.

AR run: CAT ek-ek step se banao

Pichhle step se: P1 se C predicted hai; ab model ke apne guessed C ko context mein use karenge.

Four steps poore dekho

Step 1: BOS + image -> C. Step 2: BOS,C + image -> A. Step 3: BOS,C,A + image -> T. Step 4: BOS,C,A,T + image -> EOS. EOS milne par typed answer CAT hai.

BOS	>	C
BOS C	>	A
BOS C A	>	T
BOS C A T	>	EOS

Is style ka naam AR

Previous guesses use karke next guess predict karna **autoregressive**, short **AR**, hai. Har step mein highest class pick karna **greedy decoding**. Multiple candidate strings consider karne wala alternative **beam search** hai; shown reference greedy path beam search nahi.

Reuse aur wrong guess

Encoder image ko ek baar process karta hai; z har decoder step mein reuse hota hai. Agar C wrong guessed ho, later context bhi wrong ho sakta hai. Default reference decode_ar=true aur refine_iters=1 use karta hai: AR ke baad one refinement round; refinement aage samjhenge.

Ab hamare paas: Ek full AR reading run se CAT + stop mila.

Agla page: Training mein context ke letters kahan se aate hain, woh explain karenge.

Khud check: AR ke har letter par encoder ko full CAT photo dobara process karni hai?

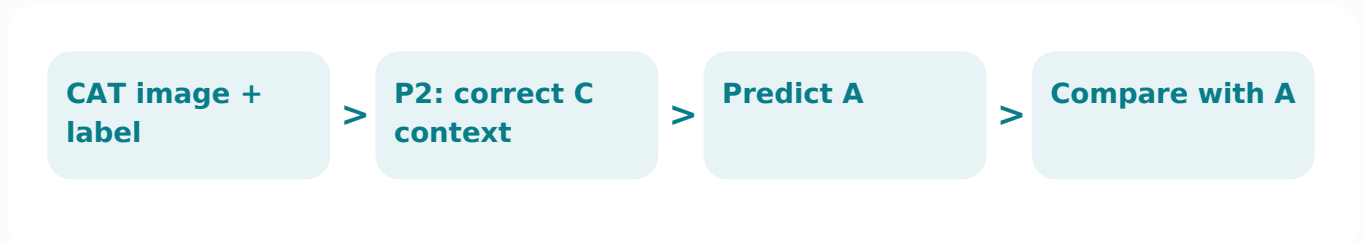
[Jawab >](#)

Training mein correct context dete hain

Pichhle step se: Inference mein CAT model ke guesses se bana; ab usi CAT example ki learning stage dekhte hain.

Ground truth ka meaning

Training label ka correct answer **ground truth** hai: CAT. Context mein model ke guesses ki jagah correct C ya A dena **teacher forcing** hai. Model ko allowed clues se desired next character predict karna sikhate hain.



Second slot example

P2 ko BOS + correct C context milega aur CAT image memory bhi. Target answer A hai. P2 ko correct A directly context mein de diya toh answer copy kar sakta hai. Apna target character allowed nahi hona chahiye.

Image aur label ki jobs

Image encoder visual evidence banata hai. Label se correct context aur targets milte hain. Predictions ko target se compare karke weights train hote hain. Training mein correct text use karna expected hai; real inference mein full correct word pehle se available nahi hota.

Ab hamare paas: Training context aur inference context ka source alag samajh aa gaya.

Agla page: PARSeq isi correct word par different allowed prediction orders sikhata hai.

Permutation: CAT ki order practice

Pichhle step se: Training image CAT aur correct characters C,A,T available hain; target apna answer copy nahi karega.

Ek alternate order lo

Ordinary order 1,2,3 hai. Ab **2,1,3** lo: pehle A ke slot ko predict karna train karo; phir C ke slot ko; phir T. Target positions hamesha slot1=C, slot2=A, slot3=T hi hain.

P2 + BOS + image	>	A target
P1 + BOS,A + image	>	C target
P3 + BOS,C,A + image	>	T target

Kis slot ko kya dikhega?

P2: BOS + image, target A. P1: BOS + correct A + image, target C. P3: BOS + correct C,A + image, target T. Different available text contexts ko same weights se use karna seekha jaata hai.

Naam aur effect

Positions ka alternate ordering **permutation** hai. **Dependency** ka meaning hai prediction ko kaunse other characters par depend karne dena. Order change hua, CAT ki spelling ACT nahi hui. Encoder ka image memory same reh sakta hai; training context rules change hote hain.

Ab hamare paas: CAT ke liye alternate allowed-context training example samajh aa gaya.

Agla page: Mask table se exactly enforce karenge kis target ko kaun dikh raha hai.

Mask: dekhne ki permission ka table

Pichhle step se: Order 2,1,3 mein A first, C second, T third training targets hain.

Table ka meaning

Mask sources ki visibility rule hai. Neeche YES matlab context character allowed. P2/A row mein sirf BOS. P1/C row mein BOS,A. P3/T row mein BOS,C,A. Image evidence teenon ko available hai; table sirf text context ka hai.

Target	BOS	C	A	T
P2 / A	YES	--	--	--
P1 / C	YES	--	YES	--
P3 / T	YES	YES	YES	--

Code convention check karo

Illustration ka YES allowed hai. Official Boolean mask mein **True blocked** ho sakta hai; Boolean matlab true/false rule. Additive mask scores mein 0 allowed aur negative infinity blocked add karta hai. Negative infinity mathematical unbounded negative value hai, normal finite number nahi.

Own answer aur indirect leakage

P2 ko A kabhi direct mat dikhao. Deeper decoder mein other context vector A ki hidden information carry kare toh indirect answer leakage bhi ho sakti hai. Isi liye implementation content aur query masks banati hai. PAD positions ke liye bhi separate masking chahiye.

Ab hamare paas: Correct text visibility enforce karne ka rule; image pathway ab bhi open hai.

Agla page: Predictions par loss calculate karke weights kaise update honge, woh dekhenge.

Khud check: 2,1,3 order mein C predict karte waqt T visible hai?

[Jawab >](#)

Loss se actual learning update

Pichhle step se: Allowed-context CAT predictions aur unke correct targets available hain.

Wrong answer ka numerical penalty

Training ki error penalty **loss** hai. Correct target ko probability p dene par **cross entropy** loss $-\ln(p)$ hai. Natural log $\ln$ ek math function hai. A ko $p=0.9$: about 0.105 penalty; $p=0.1$: about 2.303. Correct answer ka low score zyada punish hota hai.

Correct targets

>

Loss

>

Gradients

>

Weights update

Gradients aur optimizer

Weight change se loss kis direction mein kitna change hoga, us sensitivity ko **gradient** kehte hain. **Backpropagation** gradients calculate karta hai. **Optimizer** unse weights update karta hai. Update step size **learning rate** hai.

PARSeq training detail

perm_num=6 training order count ki reference setting hai; short words mein unique orders limited ho sakte hain. Image encoder once, decoder masks ke saath repeated work. PAD loss ignored. EOS specially handled: official code first two permutations ke baad EOS loss supervise nahi karta. Phone par six models deploy nahi hote.

Ab hamare paas: Shared model weights different available contexts par learning se adjust hote hain.

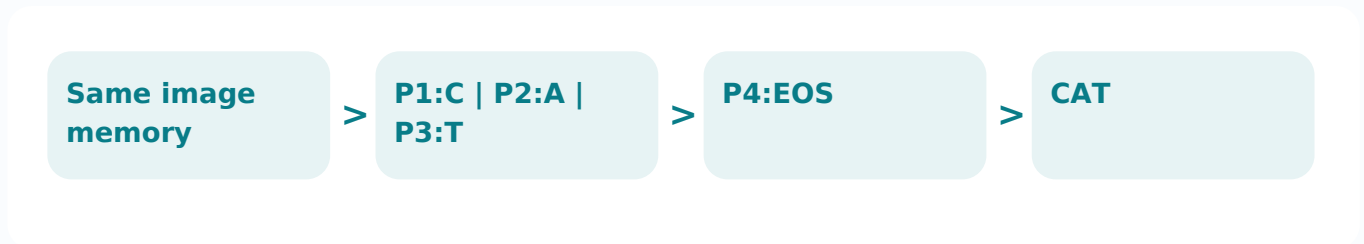
Agla page: Ab trained weights ko parallel first-guess reading mein use karenge.

NAR: CAT ke slots ko saath guess karo

Pichhle step se: Permutation-trained checkpoint available hai; AR ka sequential reading path samajh chuke hain.

First round mein neighbouring guesses nahi

P1, P2, P3 aur stop slot ko BOS context + same image memory do. P2 ko predicted C ka wait nahi karna. Saare output positions ek round mein predict hote hain. Is style ka naam **non-autoregressive**, short **NAR**.



Toy CAT run

Model image features dekhkar P1=C, P2=A, P3=T, P4=EOS nikaal sakta hai. First EOS tak string banegi. Same image encoder already patch information global mix kar chuka hai; NAR ka matlab har slot ko bas isolated patch dikhna nahi.

Same weights, different process

decode_ar=false NAR setting hai. Permutation-trained checkpoint multiple context situations ke liye learned hai. Forward-only training wale checkpoint ko NAR par switch karke equal performance assume mat karo. NAR speed/quality actual examples par compare karna hai.

Ab hamare paas: Parallel first reading se CAT aa sakta hai; galat first guess bhi possible hai.

Agla page: Ussi CAT photo ka imperfect guess revise karne ka round dekhenge.

Refinement: C4T ko CAT kar paayega?

Pichhle step se: Wahi CAT photo NAR mein toy guess C4T de gayi; second letter wrong hai.

Image + baaki letters dobara use karo

Previous guesses context banao. P2 ko C _ T aur same image memory dikhao; uska own old 4 hide karo. Model A ka stronger score nikaal sakta hai. Guess ko dobara revise karne wale round ko **refinement** kehte hain.

Previous: C4T -> P2 reads C _ T + image -> A?

Next round may be CAT. Yeh toy possible correction hai, guaranteed result nahi.

Sab old round ko padhte hain

All slots same previous C4T se update hote hain. P3 ko isi pass ka freshly fixed A nahi dikhta. Is simultaneous old-state update ko **synchronous** kehte hain. Apna old character hide karne wala rule **cloze mask** hai.

Index aur EOS ka detail

Context columns BOS,C,4,T hain; output rows C,4,T,EOS. P1 ka own C context column2 mein hai, kyunki BOS extra column hai. Mask offset check karo. EOS aur later context hide hote hain. More rounds always better nahi: model rare correct strings ko wrong bhi revise kar sakta hai.

Ab hamare paas: Refinement ke baad CAT possible; all slots previous iteration ko read karte hain.

Agla page: Ab full model route ko ek connected summary mein jodenge.

Khud check: Isi refinement round mein P3 newly corrected P2 ko read karega?

[Jawab >](#)

Start-to-end CAT journey ek saath

Pichhle step se: Pixels, encoder, decoder, training orders aur reading modes sab same CAT example par dekhe.

Image half ko jodo

CAT crop -> [1,3,32,128] normalized pixels -> 128 patches -> each 384-value embedding -> add patch locations -> 12 visual layers -> image memory z [1,128,384]. Image half ka result features hai, string nahi.



Text half ko jodo

Output-position query -> allowed text context attention -> image-memory attention -> MLP -> classifier -> scores -> chosen character ID. AR mein guessed characters next context bante hain. NAR first slots saath predict karta hai. Refinement previous guesses revise karta hai.

PARSeq ki identity

Training permutations different allowed contexts sikhati hain. Mask prevents own-answer copying. Encoder features aur decoder context dono final choices ko support karte hain. Model image ambiguity always solve karega, yeh guarantee nahi; testing se actual behavior samajhna hai.

Ab hamare paas: Full mechanism clear; ab same CAT par model-size, code aur device experiments compare kar sakte hain.

Agla page: Tiny/Small aur input-geometry changes ko isi route ke andar locate karenge.

Tiny vs Small: exactly kya badla?

Pichhle step se: Abhi Small ka route 128 visual tokens, D=384 aur 12 encoder layers use kar raha hai.

Size change ko dimensions se samjho

Model family ka version **variant** hai. Small D=384, Tiny D=192. Dono common setups mein image 32x128, patches 4x8, 128 tokens. Encoder 12 layers; decoder 1 layer. Heads encoder Small6/Tiny3, decoder Small12/Tiny6.

Small: [1,128,384] | Tiny: [1,128,192]

Same CAT crop; model representation ki width different.

CAT same, numerical width different

Tiny mein har token shorter vector hai. **Architecture** parts aur connections ka design hai. **Matrix** rows/columns mein numbers ki table hai. Width halve karne par many learned matrices smaller hoti hain, lekin total speed exactly same factor se change nahi hoti. Accuracy bhi compare karni hai.

Input geometry alag axis

Official parseq_patch16_224 experiment 224x224 image, 16x16 patches use karta hai: 14x14=196 tokens. Yeh automatically larger-width Base model nahi. AR/NAR mode, model size aur image size separate choices hain. Existing checkpoint ko changed shape par bina adaptation/validation plug mat karo.

Ab hamare paas: Experiment identity: weights + model size + input geometry + reading mode + refinement count.

Agla page: Reference code mein CAT run aur preprocessing ko locate karenge.

Python mein CAT ka baseline run

Pichhle step se: Small pretrained model aur expected image preparation ka meaning clear hai.

Code ke naye words

Python programming language; **PyTorch** model calculations ki library; **pretrained** already-trained weights. **Baseline** pehla trusted run. **Preprocessing** model input se pehle resize, colour, tensor aur normalization steps hain.

```
m = torch.hub.load('baudm/parseq', 'parseq',
                    pretrained=True).eval()
tf = SceneTextDataModule.get_transform(m.hparams.img_size)
x = tf(Image.open('cat.png').convert('RGB')).unsqueeze(0)
with torch.inference_mode():
    scores = m(x)
    text, probs = m.tokenizer.decode(scores.softmax(-1))
```

Reference workflow

Neeche compact workflow official repository ke installed environment ke liye hai. `tf` image transform hai, `x` input tensor, `scores` output tensor. RGB conversion colours match karta hai; `unsqueeze(0)` one-image batch direction add karta hai.

Mode aur output

`eval()` evaluation mode set karta hai. **Dropout** training mein random intermediate values drop karta hai; eval mein off. `inference_mode` training-gradient history avoid karta hai. Tokenizer IDs ko text mein map karta hai. Scores par character softmax lagkar decoding hoti hai.

Ab hamare paas: CAT image ka reference text aur scores save karne ka baseline pathway.

Agla page: Training data aur file locations se run ko reproducible banayenge.

Files aur dataset ka practical map

Pichhle step se: cat.png se reference input tensor aur output text/scores nikaalne ka workflow hai.

Code kahan check karna hai

model.py: encode/decode/AR/refinement. **system.py:** permutations aur loss.

modules.py: attention, query/content processing. **data/module.py, data/utlis.py:** transform aur token IDs. **Repository / repo** in code files ka online folder hai.

Train: learn

>

Validate: choose

>

Test: estimate

Data ki roles

Examples collection **dataset** hai. Train set weights learn kare; **validation** settings/checkpoint choose kare; held-out **test** final quality estimate kare. Same CAT photo ke near-duplicate versions train/test dono mein hon toh result misleading ho sakta hai. Source document/session ke hisaab se split karo.

Apne data par adapt

Pretrained weights ko naye data par train karna **fine-tuning**. Realistic blur/lighting variations **augmentation**. CAT ka T crop se remove karke label CAT rakhna wrong hai. Hindi ke visible akshar multiple Unicode code points, yani text numbers, use kar sakte hain; **tokenization** unit-splitting rule consistent rakho.

Ab hamare paas: Correct labels, honest data split aur code identity ke saath reproducible CAT evaluation.

Agla page: Prediction wrong hui toh kitni wrong, uska exact measurement dekhenge.

CAT ko C4T padhne par accuracy kya?

Pichhle step se: Wahi CAT photo ke correct label CAT aur predicted text C4T available hain.

Whole word ya character edits?

Measurement rule **metric** hai. **Exact-word accuracy** fully correct words/total words. Is one sample par 0%, kyunki C4T exactly CAT nahi. **Character Error Rate, CER** = replacements+deletions+insertions divided by reference character count. One replacement/3 = **33.3%**.

Reference CAT -> prediction C4T

Exact match = 0% | CER = 1/3 = 33.3%

Confidence ko test karo

Output scores se result trust estimate **confidence** ban sakti hai. High score always correct nahi. **Threshold** accept/reject cut-off hai. **Coverage** accepted/all, aur acceptance precision correct-accepted/accepted hai. Validation examples par cut-off choose karo, held-out results report karo.

Only CAT test sufficient nahi

CAT se process samjha, real benchmark ko varied words/scripts chahiye. **Benchmark** fixed examples/rules ka evaluation hai. Case, punctuation, blur, long labels aur digits separately report karo. Small quality improvements sample noise bhi ho sakte hain; paired examples compare karo.

Ab hamare paas: CAT result ka quality score clear; same quality checks deployment changes par repeat honge.

Agla page: Reference model ko phone runtime ke format mein le jaayenge.

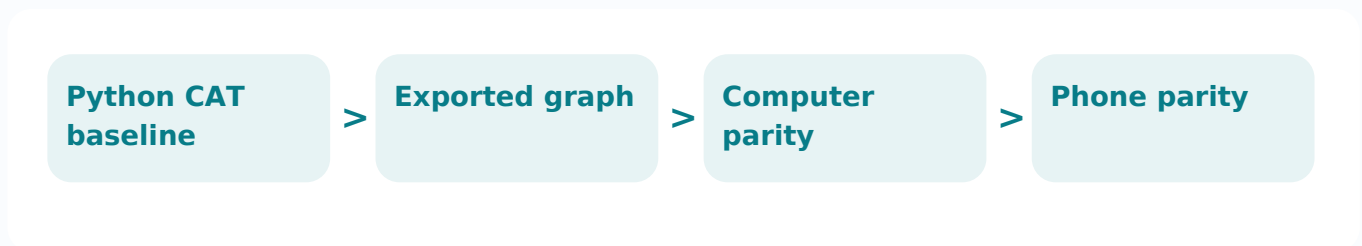
Export: wahi CAT, doosra runner

Pichhle step se: Python baseline CAT input/output aur evaluation rules saved hain.

Naye names ka purpose

Export model ko different runner-compatible representation mein convert karna hai.

Graph connected calculations; **operator** one operation, jaise multiply. **Runtime** graph chalane ka software. ONNX model format hai; ONNX Runtime uska runner. LiteRT doosra on-device runner hai.



Simple first experiment

Fixed-size NAR with refine_iters=0 ek useful first export candidate hai, kyunki character-by-character loop remove hota hai. Candidate matlab test karne wala choice, guaranteed success nahi. Converter ke supported operators aur required input/output shapes verify karo.

Parity ladder

Parity same behavior ka milaan. CAT ke exact input tensor compare karo; phir encoder z, logits, IDs, EOS aur text. Ek random tensor check enough nahi. AR loop graph mein supported control flow se ya app ke repeated decoder calls se chal sakta hai; copies/calls ka cost measure karo.

Ab hamare paas: Exported model ko reference CAT behavior se compare karne ka concrete plan.

Agla page: Phone par CPU/GPU/NPU mein graph actually kahan execute hua, woh dekhenge.

Khud check: Exported file load ho gayi. Kya CAT correctness prove ho gayi?

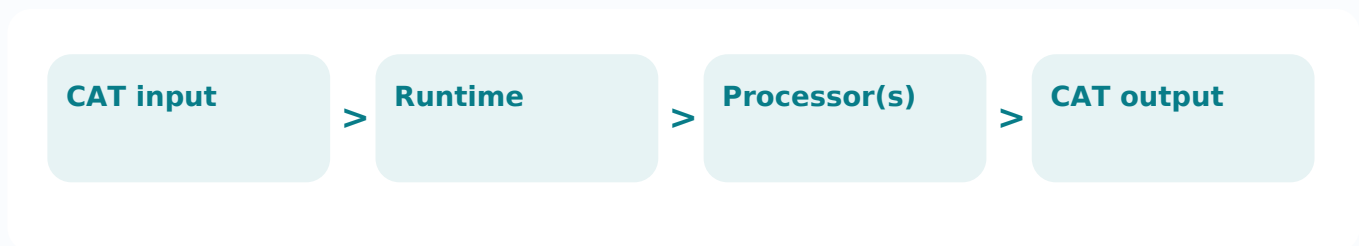
[Jawab >](#)

Phone par calculations kahan chalti hain?

Pichhle step se: Exported model ka CAT behavior reference se compare ho sakta hai.

Processors ke roles

CPU general processor; **GPU** many parallel computations; **NPU** neural-model specialised processor. **Backend** runtime ka execution route. Supported graph parts ko processor par bhejne ka mechanism **delegate** ya execution provider kehla sakta hai.



NPU label alone enough kyun nahi

Unsupported operators CPU par run ho sakte hain. Alternate route **fallback** hai. Graph ka divided region **partition**. Data transfer aur waiting total time badha sakte hain. Actual execution record **trace** se placement check karo; supported chip/runtime combination matters.

CAT ko app mein run karo

Working memory **RAM**, temporary data area **buffer**. Screen interaction ka **UI thread** heavy inference se block mat karo. Pending frames ki line **queue** bound rakho. Reuse loaded model. Android NNAPI neural interface deprecated in Android15 hai; actual supported current path choose karo.

Ab hamare paas: Phone par correct CAT result ke saath actual processor placement verify karne ka plan.

Agla page: Weights ka number format chhota karke quality aur storage compare karenge.

Quantization: rounding se CAT badlega?

Pichhle step se: Phone-run model available hai; weight storage aur numerical format optimise karna hai.

Number format ki vocabulary

Precision representation ki detail hai. **Bit** 0/1 unit; 8 bits=1 byte. **FP32** 32-bit floating, **FP16** 16-bit floating, **INT8** 8-bit integer format. Limited numerical levels se approximate karna **quantization**.

Same CAT -> FP32 result CAT -> INT8 result CAT or C4T?

Paired inputs par score drift aur final string dono compare karo.

Concrete rounding aur storage

0.02 ke steps mein 0.37 ko 0.38 approximate karna 0.01 change hai; actual tie-rounding rule verify karo. Illustrative 24M weights: FP32=96 MB, FP16=48 MB, INT8=24 MB; MB million bytes. Extra runtime memory excluded; named checkpoint count ka claim nahi.

CAT ke same scores compare karo

A aur 4 scores close hon toh rounding top choice flip kar sakti hai. **Calibration** representative inputs se ranges choose karna. **PTQ** training ke baad quantization; **QAT** training mein quantization effects simulate karna. **Mixed precision** sensitive parts ko more precise format mein rakhna. Smaller file speed guarantee nahi.

Ab hamare paas: Number-format experiment ki quality/storage checks clear.

Agla page: Ab same CAT workflow ka total time measure karenge.

Total speed: sirf decoder time nahi

Pichhle step se: CAT ko correct read karne ke multiple modes/formats hain; ab unka cost compare karna hai.

Measurements define karo

Latency ek result ka time; **throughput** per time results count. **Profiling** stage-wise time/resources measure karna. **Cold** first load/run; **warm** already ready model. p50 median, p95 timing value jiske andar about 95% runs finish hote hain.

43 ms total -> decoder 6 to 3 ms -> 40 ms total

Paired quality + total time + memory + sustained behavior compare karo.

Toy CAT timing

Detect18 + crop4 + encoder12 + decoder6 + text handling3 = 43 ms. ms millisecond, second ka thousandth part. Decoder half to 3 ho toh total 40 ms, half nahi. Speedup $43/40=1.075x$. Yeh teaching timings hain, measured phone results nahi.

Sustained use aur later options

Heat se frequency lower hona **thermal throttling**. Long continuous runs, RAM aur energy bhi test karo. Asynchronous call actual finish se pehle return kar sakti hai; timer se pehle completion wait. **Distillation** teacher model se smaller student training; **structured pruning** whole channels/layers remove karna. Har change par quality retest karo.

Ab hamare paas: CAT result ki correctness ke saath real feature cost measure karne ka method.

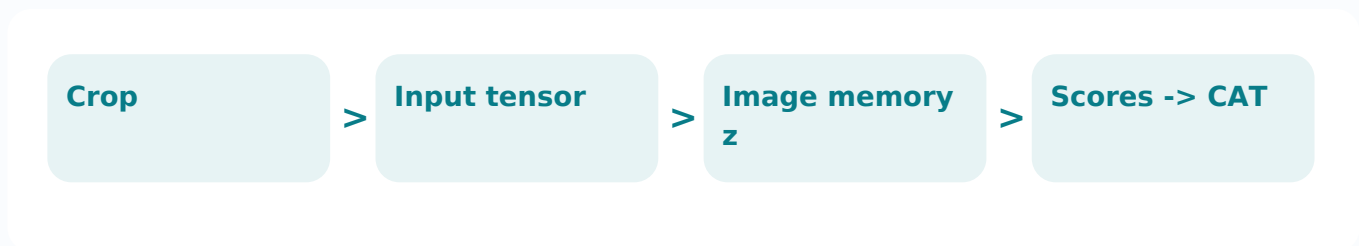
Agla page: Final page par wrong result ko earliest broken step tak trace karenge.

CAT wrong hua: chain mein break dhoondo

Pichhle step se: Poora photo-to-text route aur quality/time measurements clear hain.

First wrong stage locate karo

Output C4T aaya. Exact crop dekho: A visible hai? Input tensor reference se match? Encoder z different? Decoder scores A se4 par flip? EOS early? Galti ke first visible point par test karo. **Hypothesis** testable possible cause hai; ek baar mein ek change karo.



Regression aur reproducibility

Fixed crops jo har change ke baad run hon **regression set** hain. Checkpoint ka **hash** identifying file fingerprint; version saved revision. Config settings, charset IDs, image prep, reading mode, rounds, precision, runtime/device sab record karo.

Final explanation apne words mein

CAT crop ke pixels learned patch vectors bane; position added; encoder attention/MLP se visual memory bani. Decoder output slots ko allowed text + image mila; scores se letters/EOS nikle. Permutations training context change karti hain. AR sequential, NAR parallel, refinement previous guesses revise karta hai. Yeh connected PARSeq mechanism hai.

Ab hamare paas: Final standalone CAT-based PARSeq understanding aur practical debugging route.

Agla page: Answers aur clickable journey map se weak step par wapas jaa sakte ho.

Jawab + apni understanding check karo

Page number par tap karke us example par wapas jaa sakte ho. Sahi answer yaad karne se zyada important hai uska reason apne words mein bolna.

- 02** Pehle exact crop dekho. Model ko missing half-letter ka proper evidence mila hi nahi.
- 06** Nahi. Patches image ke areas hain; ek character kai patches mein ho sakta hai.
- 09** Nahi. Is operation mein source ki information ka contribution 0.75 hai; character correctness alag hai.
- 12** Nahi. 128 processed image-token vectors hain. Letters decoder ko predict karne hain.
- 17** Nahi. Image memory z once compute karke decoder steps mein reuse ho sakti hai.
- 20** Nahi. C ko BOS aur A allowed hain; T later position in the sampled order hai.
- 23** Nahi. All slots previous iteration ke predictions ko read karke new results saath banate hain.
- 29** Nahi. Input, features, scores, EOS aur final string ka reference se comparison baaki hai.

5 KAAM JO TUM KHUD EXPLAIN KAROGI

- ☐ Pixels se patch vectors tak ki journey.
- ☐ Attention mein score aur mixed information ka farq.
- ☐ Encoder se decoder tak kya pass hota hai.
- ☐ CAT ke AR, NAR aur refinement steps.
- ☐ Permutation mein spelling same kaise rehti hai.

Journey map + original sources

Yeh naya standalone final edition hai. Har chapter wahi CAT image follow karta hai. Chhote numerical examples teaching illustrations hain; actual checkpoint ke measured features nahi.

- | | |
|--|---|
| 01 Ek hi CAT photo, start se end tak | 02 Pehle CAT wali jagah nikaalo |
| 03 CAT ki image ab colour numbers hai | 04 Tensor: numbers ka organised dabba |
| 05 Calculations ko C pehchanna kisne sikhaya? | 06 CAT ko patches mein baanto |
| 07 Patch ke numbers se naya vector banao | 08 Vector ko uski jagah bhi batao |
| 09 Attention: ek patch ko extra clue do | 10 Q, K, V: weights kaise decide hue? |
| 11 Ek processing block ka poora kaam | 12 Encoder output: exactly kya mila? |
| 13 Output slots: text kahan likha jaayega? | 14 C, A, T aur start/stop ke IDs |
| 15 Decoder ko do sources milte hain | 16 Scores se first character C choose karo |
| 17 AR run: CAT ek-ek step se banao | 18 Training mein correct context dete hain |
| 19 Permutation: CAT ki order practice | 20 Mask: dekhne ki permission ka table |
| 21 Loss se actual learning update | 22 NAR: CAT ke slots ko saath guess karo |
| 23 Refinement: C4T ko CAT kar paayega? | 24 Start-to-end CAT journey ek saath |
| 25 Tiny vs Small: exactly kya badla? | 26 Python mein CAT ka baseline run |
| 27 Files aur dataset ka practical map | 28 CAT ko C4T padhne par accuracy kya? |
| 29 Export: wahi CAT, doosra runner | 30 Phone par calculations kahan chalti hain? |
| 31 Quantization: rounding se CAT badlega? | 32 Total speed: sirf decoder time nahi |
| 33 CAT wrong hua: chain mein break dhoondo | |

ORIGINAL SOURCE = ASLI RESEARCH / CODE

PARSeq research paper

Official model, training aur data code

ONNX Runtime mobile / deployment guidance

LiteRT model conversion / phone execution

Research paper researchers ka method/result document hai. Repository online code folder hai. Phone timing/storage calculations examples hain; Samsung internals ya measured device speed ka claim nahi. Architecture details previously checked official sources par based hain.